

Instruction TLB: Fully associative. 32 entries.

Data TLB: Fully associative. 32 entries.

L1 Data cache:

Size: 64Kb 2-way associative.
lines per tag=1 line size=64 bytes.

L1 Instruction cache:

Size: 64Kb 2-way associative.
lines per tag=1 line size=64 bytes.

L2 (on CPU) cache:

Size: 512Kb 8-way associative.
lines per tag=1 line size=64 bytes.

The physical package has 2 cores

WARNING: Detected SMP, but unable to access cpuid driver.

Used Uniprocessor CPU routines. Results inaccurate.

You can see my CPU has an L1 (data cache) of 64 KB, an L1 (instruction cache) of 64 KB and an L2 cache of 512 KB. This information is very useful when tuning your CPU—which we'll do in later sessions.

dumpe2fs

This tool displays very crucial filesystem information like volume, name, state, block size, etc. This information is very useful while tuning RAID and mounting external journals. These topics will be covered in detail in the upcoming sessions.

The following is the output of `dumpe2fs /dev/sda1` which lists the details of my `/dev/sda1` file system:

```
Filesystem volume name: <name>
Last mounted on: <not available>
Filesystem UUID: 515d8a09-85e4-45a7-bc77-3f7336234275
Filesystem magic number: 0xEF53
Filesystem revision #: 1 (dynamic)
Filesystem features: has_journal ext_attr resize_inode dir_index
filetype needs_recovery sparse_super
Filesystem flags: signed_directory_hash
Default mount options: user_xattr acl
Filesystem state: clean
Errors behavior: Continue
Filesystem OS type: Linux
Inode count: 51200
Block count: 204800
Reserved block count: 10240
Free blocks: 124082
Free inodes: 51132
First block: 1
Block size: 1024
Fragment size: 1024
Reserved GDT blocks: 256
Blocks per group: 8192
Fragments per group: 8192
Inodes per group: 2048
Inode blocks per group: 256
```

```
Filesystem created: Sat Jul 25 11:46:19 2009
Last mount time: Tue Aug 18 11:28:43 2009
Last write time: Tue Aug 18 11:28:43 2009
Mount count: 53
Maximum mount count: -1
Last checked: Sat Jul 25 11:46:19 2009
Check interval: 0 (<name>)
Reserved blocks uid: 0 (user root)
Reserved blocks gid: 0 (group root)
First inode: 11
Inode size: 128
Journal inode: 8
Default directory hash: half_md4
Directory Hash Seed: 82c0cbea-44f5-491f-b200-1370950b487f
Journal backup: inode blocks
Journal size: 4113k
<-output truncated->
```

Graphical reporting of data

Till now we have gathered enough data from our systems. We will now plot this data graphically to analyse it in a better way. The tools that we will use here are **gnuplot** and **mrtg** for graphical reporting.

gnuplot

gnuplot is a plotting tool that we can operate in one of two modes:

1. Interactive mode: Here you issue a command at the **gnuplot** prompt to adjust a graph to your liking.
2. Batch mode: Here you can feed **gnuplot** the commands from a file to make it generate graphs in a batch.

Let's gather some data first using the **sar** command. Then we'll use the **gnuplot** command to visualise the data. Run `sar -r 2 100 > ~/meminfo` & to collect the data and store it in a file named **meminfo**. Remember to set alias `sar="LANG=C sar"` so that you have outputs in the 24-hour clock format. Figure 6 shows the output from the `~/meminfo` file.

Now, use a text editor (for example, **vi**) to open the file and remove any non-numeric lines—which basically means the first three lines of Figure 6 need to be deleted.

Make sure you have **gnuplot** installed. If it is not installed, use your distro's package manager or download it from www.gnuplot.info.

Create a file using a text editor with the following lines:

```
set data time
set timefmt "%H:%M:%S"
set xlabel "Time"
set ylabel "Memory (in Kb)"
plot "meminfo" using 1:2 title "FREE" with lines
replot "meminfo" using 1:5 title "BUFFERED" with lines
```